N-Flight Planning Webservice Technical Guide

SERVICE INTEGRATION POLICIES & FUNCTIONAL OVERVIEW

-

SCOTT SEAGER

2.28 - 12.11.2024

NAVBLUE
AN AIRBUS COMPANY

Table of Contents

1. About this Document ... 4
   1.1. Purpose ... 4
   1.2. Scope ... 4
   1.3. Owner ... 4
   1.4. Distribution Method ... 4
   1.5. Amendment Procedure ... 5
   1.6. Intended Audience ... 5
   1.7. Conventions ... 5
   1.8. Definitions, Terminology, and Acronyms ... 6
   1.9. Assumptions, Risks, and Mitigations ... 8
2. History ... 9
   2.1. Document ... 9
   2.2. Service Functionality ... 12
3. Introduction ... 23
   3.1. Overview ... 23
   3.2. History ... 24
   3.3. Methodology ... 24
   3.4. Migration ... 25
   3.5. Designated Contacts ... 25
   3.6. First Time Recipients ... 26
4. Access ... 27
   4.1. In-House Installation ... 27
   4.2. Hosted Service ... 28
   4.3. Motivation ... 29
   4.4. Fail-Over ... 29
5. Interface ... 30
   5.1. Rights Management ... 30
   5.2. XML Namespace ... 31
   5.3. NavWeb:User ... 31
   5.4. NavWeb:MapElements ... 31
   5.5. NavWeb:FlightId ... 32
   5.6. NavWeb:FormatInput ... 32
      5.6.1. XML Output ... 33
      5.6.2. Text & PDF Output ... 33
   5.7. NavWeb:EffFormatInput ... 33
   5.8. Recall Numbers vs. Serial Numbers ... 34
6. SOAP Service Methods ... 36
   6.1. AddWeather ... 36
   6.2. BuildRoute ... 36
   6.3. ComputeFlight ... 37
   6.4. Crew ... 39
   6.5. DecodeRoute ... 39
   6.6. Defaults ... 40
   6.7. EstimateFlightCost ... 40
   6.8. FlightStatus ... 41

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

6.9. GetAircraft ... 41
6.10. GetAirport ... 41
6.11. GetAlternateList ... 41
6.12. GetFlightInfo ... 41
6.13. GetFlightList ... 42
6.14. GetServiceInfo ... 42
6.15. GetSidStarList ... 42
6.16. GetSupportedAirports ... 42
6.17. GetWxChartInfo ... 42
6.18. GetWxCharts ... 43
6.19. ImportFlight ... 43
6.20. MelCdl ... 44
6.21. RawWeatherNotam ... 44
6.22. RecallFlight ... 45
6.23. RouteAnalysis ... 46
6.24. WeatherNotam ... 46
6.25. WindReport ... 46
7. WMS Service Addendums ... 47
7.1. FILTERS ... 47
7.2. INFOX / INFOY ... 47
7.3. RADIUS ... 47
7.4. WRAP ... 47
7.5. QUERYTIME ... 48
8. SOAP Recipes ... 49
8.1. Adding/Updating A Flight ... 49
8.2. Retrieve Service Version And Change Descriptions ... 49
8.3. Generating Default Operational Flight Plan Format ... 50
8.4. Generating ARINC 633 XML Flight Plan ... 50
8.5. Generating Multiple Output Products ... 51
8.6. Retieving MEL/CDL Dictionary Information ... 52
8.7. Retrieving Flight Summaries Assigned To A Group/Desk ... 53
8.8. Retrieving Basic Flight Information For Multiple Flights ... 53
9. WMS Recipes ... 54
9.1. Showing Only Country Boundaries ... 54
9.2. Overlaying A Flight Plan Route ... 54
9.3. Overlaying Multiple Routes ... 55
9.4. Overlaying Wind Barbs ... 55

Tables of Figures

Figure 1: Two-Stage Output Field Addition ... 25
Figure 2: In-House Data Flow ... 27
Figure 3: Hosted Service Data Flow ... 28
Figure 4: Recall vs. Serial Number Generation ... 35

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

1. About this Document

This document acts as a supplemental resource to third-party development staff to assist with the service-based integration of their products with the N-Flight Planning system.

1.1. Purpose

There is a certain degree of technical foundation knowledge that is required to help third-party developers communicate with *N-FP*'s webservices. This has typically been relayed via personal correspondence without a formal document to act as a distributable outline. This document addresses that shortcoming.

1.2. Scope

This document addresses the higher-level concepts and functionality present within the N-FP *SOAP* and *WMS* services, and provides the base-level knowledge necessary to communicate with them. It does not include documentation of low-level data structures or method signatures, nor does it provide a thorough description of the behaviours and semantics of those method calls.

In regards to SOAP functionality, that information is more aptly distributed via the related *WSDL* file because:

- It is automatically generated by the source code build system based on the functionality present in source control at that time, ensuring that it stays current.
- It is packaged and installed as part of the N-FP product on both the *back-end* or *engine server*, as well as the *front-end* or *web server*, making access to it easy and ensuring that its content matches the functionality present on those systems.
- The WSDL file is annotated with per-method and per-field documentation, which can be included with auto-generated code and class definitions by modern development environments.

In regards to WMS functionality, this is standardized by the Open Geospatial Consortium and details can be found at http://www.opengeospatial.org.

1.3. Owner

This document is owned by the Software Development Manager for Flight Planning.

1.4. Distribution Method

The primary method of internal distribution of this document shall be in electronic format via internal publication services. Where differences exist, paper copies are considered subordinate to electronic formats.

Since customers, or others acting on their behalf are the primary consumers of this document, other distribution methods such as email are expected and permitted. Note this document's confidentiality.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

1.5. Amendment Procedure

Update requests will be made to the Software Development Lead for Flight Planning, and those updates will be executed by them or a delegate of their choosing.

Regular reviews are not performed since content will remain applicable for all future versions of the service offering, though that content should be augmented and/or refined based on input from the reader as necessary.

1.6. Intended Audience

Third-party software development staff can use this document to assist them in understanding how their own systems and products can interface with N-FP's webservices, as well as receive a general overview of the functionality provided therein.

1.7. Conventions

Italics
: Used to bring attention to new terms or add emphasis.

`Fixed Width`
: Highlights filenames, pathnames, or literal values.

`Blue Fixed Width`
: Used to show an actual source code-level data type, class, module, method, or repository.

`Grey Background Fixed Width`
: Shows literal source code snippets, configuration file contents, or data files.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

1.8. Definitions, Terminology, and Acronyms

Term	Definition
AIRAC	Aeronautical Information Regulation And Control. Within the context of N-FP, this represents the version of navigation data, which changes every 28 days, that should be utilized for a given service request.
ALC	Airline Code.
API	Application Programming Interface. A set of data structures and subroutines offered by a system to export functionality to others.
ASD	Aircraft Situational Display.
B2B	Business-to-business.
Back-End	The service-layer machine responsible for the “business logic” that comprises N-FP’s functionality.
DispatchProxy	An N-FP service running on the web server that is tasked with routing SOAP requests for hosted customers to the engine within a protected area of the network, and shuttle the responses back accordingly.
DR	Disaster Recovery.
DMZ	De-Militarized Zone. A network design component where a set of services sit within a relatively unprotected area that can be accessed from the outside, and have only limited or minimal access to internal portions of the network.
EFB	Electronic Flight Bag. An application allowing display and interaction with digital flight documents.
EFF	Electronic Flight Folder. A standardized archive of documents in various formats applicable to a particular flight, and made available through an electronic flight bag application.
Engine Server	See back-end.
FIR	Flight Information Region. An area of airspace controlled by an air traffic control center.
FOMS	Flight Operations Management System. A legacy terminal-based NAVBLUE flight planning product. Today, FOMS refers to the “back office” programs that allow users to view and modify navigation, aircraft performance, and airline configuration data. Each FOMS program has a numeric code associated with it.
Front-End	The presentation-layer machine that offers the browser-based user interface for the N-Flight Planning product.
GeoserverProxy	An N-FP service running on the web server that is tasked with routing WMS requests from the user interface to the engine and returning results. This service can also be used by customers within a hosted environment get access to geospatial data.
GUID	Globally Unique Identifier. A technically-standardized numeric value, expressed as a combination of hexadecimal digits that guarantees an infinitesimal chance of two identical values ever being generated. Also referred to as a *UUID*.
IATA	International Air Transport Association.
ICAO	International Civil Aviation Organization.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

Macro	A simple legacy query language allowing the retrieval of weather and NOTAM data based on a series of properties.
MEL/CDL	Minimum Equipment List; Configuration Defect List. Definitions of deviations from normal aircraft configuration and/or operation.
METAR	A text bulletin detailing actual observed weather, issued upon the hour.
N-FP	N-Flight Planning.
NOTAM	Notice To Air Men. Text-based notifications about circumstances that may have an operational impact on flights operating to certain stations, areas, or interim positions.
OOOI	Out, Off, On, In. Events that mark changes in aircraft location and status, coinciding with push-back from the ramp, wheels off and eventually back on ground, and completed taxi to the destination gate.
Pending Input	A value provided as part of a flight plan calculation request that has been associated with the applicable flight, but which has not yet been applied and incorporated into a computed flight plan.
SIGMET	Significant weather conditions present within a defined geographical area, such as icing, turbulence, convective thunderstorm activity, and volcanic ash clouds.
SOAP	Simple Object Access Protocol. An XML-based platform/runtime/language-agnostic communication protocol allowing programmatic communication between heterogeneous systems.
SPECI	A METAR issued outside of the regularly scheduled interval due to a significant change in conditions.
STD	Scheduled Time of Departure.
TAF	Terminal Area Forecast. Predicted weather conditions over a (typically) 30-hour period.
UFI	Universal Flight Identifier. An alternative logical flight key utilized by some third-party systems, containing an *ALC*, flight number, optional suffix, initial origin flight date, origin, destination, and optional repeat number.
URL	Universal Resource Locator. A descriptor indicating a service protocol and location for network communication.
UUID	Universally Unique Identifier. See GUID.
Web Server	See front-end.
WMS	Web Mapping Standard. A *URL* request format allowing the retrieval of graphical geospatial information.
WSDL	Web Service Definition Language. An XML-based document providing low-level technical details on exactly how two systems should structure XML communication to access functionality.
WX	Weather.
XML	Extensible Mark-up Language. An industry-standard data format allowing structured data communication and representation between computer systems.
XSD	XML Schema Definition. An XML document that enforces the structure of another XML document to match a given data contract.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

1.9. Assumptions, Risks, and Mitigations

It is assumed that the reader of this document will have the requisite knowledge necessary to understand and develop software using SOAP, WMS, and their related technologies. This is not a tutorial nor a general webservice programming "how-to" guide. Information pertaining to those needs can be readily found through online and commercial publications.

Many of the specific services offered via SOAP from N-FP require a certain degree of subject matter knowledge in the area of flight planning and/or the N-Flight Planning product in particular, which staff or contract employees of customers may not possess. Airline or NAVBLUE guidance to development staff may be necessary to bridge that gap.

Updating this document as the SOAP and WMS interfaces change requires proactive work to be done by a NAVBLUE software developer, leaving the possibility that it may drift out of sync with the implemented version present on a given production system or within the source control system. The WSDL file and WMS capabilities should always be consulted (see §4) and considered to be the authoritative definition of functionality.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

2. History

2.1. Document

Date	Rev	Description	Revised By
2015/05/25	0.1	• Initial version.	Scott Seager
2015/05/26	0.2	• Include reference to the possibility of documentation drift and the need to consult the WSDL file for an authoritative description of functionality in §1.9 and §6. • Include an explicit reference to the possibility of returning a parseable XML document containing computed flight plan data as part of the ComputeFlight() service call in §6.3.	Scott Seager
2015/09/30	1.0	• Add information about the new ImportFlight() service call in §6.19, available with N-FP 15.5. • Small language changes.	Scott Seager
2016/03/07	1.1	• Include information about NavWeb:FlightId in §5.5. • Add information about the new FlightInfo() and FlightList() service calls in §6.12 and §6.13, respectively, available in N-FP 16.1. • Supplement the description of ComputeFlight() in §6.3 to include more information about the importance of the returned serial number, as well as newly available sparse compute inputs. • Explicitly state that ImportFlight() always treats inputs sparsely in §6.19, as opposed to ComputeFlight() which is conditional on the input parameters. • Additional information about pending compute inputs in §6.19. • Include the initial version of N-FP that each service call was made available.	Scott Seager
2016/06/16	1.2	• Update FlightInfo() and FlightList() to reflect their rename to GetFlightInfo() and GetFlightList(), respectively, which occurred before production deployment. • Sections added for the Crew(), Defaults(), and RawWeatherNotam() service calls in §6.4, §6.6, and §6.21, respectively. • Include notes about the possibility for slight client modification when importing newer versions of the WSDL in §3.3. • Added information about how to retrieve XSD schemas for the MelCdl() call in §6.20.	Scott Seager

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

2016/08/24	1.3	• Add a note to §3.3 clarifying that the webservice offering is not intended to, nor does it provide complete coverage of modules for database access and manipulation.	Scott Seager
2017/04/06	1.4	• Sections added for the FlightStatus(), GetWxChartInfo(), and GetWxCharts() service calls in §6.8, §6.17, and §6.18, respectively.	Scott Seager
2017/05/18	1.5	• Branding changes. • Update WSDL URL when accessed via the DispatchProxy as of N-FP 17.1 in §4.2.	Scott Seager
2017/07/10	2.0	• Further branding changes and document rename. • Add information regarding newly available WMS webservices. • Add diagrams depicting hosted vs. on-premises N-FP system installation and access to §4. • Add service functionality history in §2.2. • Service version and functionality migration information added in §3.4. • Section added for GetServiceInfo() in §6.14. • Section added for technical contact assignment in §3.5. • Indicate the possibility/likelihood of field deprecation/removal in §5.5, §6.3, and §6.14.	Scott Seager
2017/10/16	2.1	• Functional changes in N-FP 17.5 added to §2.2. • AIRAC cycle addition to NavWeb:User in §5.3.	Scott Seager
2018/01/17	2.2	• Functional changes in N-FP 17.7 added to §2.2. • Include information about data feed semantics with RawWeatherNotam() in §6.21. • Section added to address system fail-over possibilities and impact in §4.4.	Scott Seager
2018/05/17	2.3	• Functional changes in N-FP 18.1 added to §2.2. • Added information about the EnvAlc field of the NavWeb:User structure in §5.3. • Removed a reference to the now-deprecated toggling of automation via ImportFlight() in §6.19, since this is now accomplished with FlightStatus().	Scott Seager
2018/07/30	2.4	• Functional changes in N-FP 18.3 added to §2.2.	Scott Seager
2018/09/27	2.5	• Functional changes in N-FP 18.5 added to §2.2. • Sections added for AddWeather() in §6.1 and GetSupportedAirports() in §6.16. • Section added to describe the role and use of NavWeb:FormatInput in §5.6. • Section added to provide examples of common SOAP method call “recipes” in §8.	Scott Seager

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

2019/01/18	2.6	• Functional changes in N-FP 18.7 added to §2.2. • Section added for EstimateFlightCost() in §6.7. • Additional content in §6.3 covering the possibility of side effects implicitly changing values in sparse compute scenarios. • Section added to describe the role and use of NavWeb:EffFormatInput in §5.7. • Additional content in §6.22 to provide specifics on the mechanisms by which content is generated by RecallFlight().	Scott Seager
2019/05/14	2.7	• Functional changes in N-FP 19.1 added to §2.2.	Scott Seager
2019/07/30	2.8	• Functional changes in N-FP 19.3 added to §2.2. • MEL/CDL dictionary retrieval SOAP recipe added in §8.6. • First Time Reader section added in §3.6. • Clarified that a pseudo-hosted solution approach can also be employed at customer sites to facilitate integration capabilities from third-parties outside of their private network in §4.2. • Clarified that both entity-encoding and CDATA-enclosing are valid approaches to escaping XML payloads with the Crew() and MelCdl() services in §6.4 and §6.20, respectively. • Information about discarding the results from ComputeFlight() added in §6.3.	Scott Seager
2019/09/25	2.9	• Functional changes in N-FP 19.5 added to §2.2. • References to sections §1.2 and §1.9 added to the first time recipients primer in §3.6. • Information about toggling RAIM and/or ADS-B validation functionality following flight plan calculations with ComputeFlight() has been added to §6.3.	Scott Seager
2019/12/23	2.10	• Functional changes in N-FP 19.7, as well as a missed item in N-FP 19.5 added to §2.2.	Scott Seager
2020/05/01	2.11	• Functional changes in N-FP 20.1, as well as two missed items in N-FP 19.5 added to §2.2. • Section added to provide examples of common WMS request call “recipes” in §9.	Scott Seager
2020/06/01	2.12	• Functional changes in N-FP 20.3 added to §2.2. • Added content regarding new SIGMET retrieval behaviour in RawWeatherNotam() in §6.21.	Scott Seager
2020/10/02	2.13	• Functional changes in N-FP 20.5 added to §2.2.	Scott Seager
2020/12/09	2.14	• Functional changes in N-FP 20.7 added to §2.2. • Section added for DecodeRoute() in §6.5. • IfpsAutoAssist usage information added to §6.2 and §6.3.	Scott Seager

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

2021/04/29	2.15	• Functional changes in N-FP 21.1 added to §2.2. • Information about weight unit interpretation within the NavWeb:ComputeInput structure added to §6.3 and §6.19 for ComputeFlight() and ImportFlight(), respectively. • Note added about the possibility of training and/or subject matter expert involvement being required for more complicated webservice functionality in §6.4, §6.6, and §6.20 for Crew(), Defaults(), and MelCdl(), respectively.	Scott Seager
2021/08/12	2.16	• Functional changes in N-FP 21.3 added to §2.2.	Scott Seager
2021/11/23	2.17	• New SOAP recipes illustrating the use of GetFlightList() and GetFlightInfo() as §8.7 and §8.8, respectively.	Scott Seager
2022/03/06	2.18	• Functional changes in N-FP 21.7 added to §2.2.	Scott Seager
2022/05/20	2.19	• Functional changes in N-FP 22.1 added to §2.2. • New SOAP recipe specific to ARINC 633 XML flight plan retrieval added as §8.4	Scott Seager
2022/09/09	2.20	• Functional changes in N-FP 22.3 added to §2.2. • Description of recall and serial numbers for plan revisions added in §5.8.	Scott Seager
2022/12/20	2.21	• Functional changes in N-FP 22.5 added to §2.2.	Scott Seager
2023/02/09	2.22	• Functional changes in N-FP 22.7 added to §2.2.	Scott Seager
2023/05/03	2.23	• Functional changes in N-FP 23.1 added to §2.2.	Scott Seager
2023/07/10	2.24	• Functional changes in N-FP 23.3 added to §2.2.	Scott Seager
2023/10/27	2.25	• Functional changes in N-FP 23.5 added to §2.2.	Scott Seager
2024/02/29	2.26	• Functional changes in N-FP 23.7 added to §2.2. • Values for the PackageLayoutFile property of the NavWeb:FormatInput structure, available to all users and N-FP installations, are now listed in new sections §5.6.1 and §5.6.2. • Details about crew assignment change semantics in ImportFlight() added to §6.19.	Scott Seager
2024/07/18	2.27	• Functional changes in N-FP 24.1 added to §2.2. • Corrected EFF integration keys listed in §5.6.2.	Scott Seager
2024/11/12	2.28	• Functional changes in N-FP 24.3 added to §2.2.	Scott Seager

2.2. Service Functionality

Version	Change	Description
10.5	143910	Initial service availability.
11.1	159625	Supernumary crew rank added to ComputeFlight().
	159842	Introduction of the WeatherNotam() service call.
11.3	149336	ACARS flight plan uplink ID added to NavWeb:FormatInput, allowing transmissions from ComputeFlight() and RecallFlight().
	175484	ATC flight number added to ComputeFlight().

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

	177791	Rule compliance status added to the output of ComputeFlight().
11.5	168696	Destination/Alternate IFR fuel/time added to ComputeFlight().
	169041	ATC callsign added to ComputeFlight().
	170580	Optional runway information made available from GetSidStarList().
	185868	Airway distance, great circle distance, fuel, time, and cost information returned as part of the NavWeb:Route structure.
	183525	ATC item 19 inclusion and alternate fuel/time biasing toggling within ComputeFlight().
12.1	171525	Male, female, and child passenger counts/weights available within ComputeFlight().
	185518	Per-leg performance profile overrides added to ComputeFlight().
	186269	Flight plan header section retrieval added to NavWeb:FormatInput, allowing short-format retrieval from ComputeFlight() and RecallFlight().
	185647	PNR fuel and time added to ComputeFlight().
	195528	MORA driftdown method and engine count selection added to ComputeFlight().
	135766	Alternate burn method added to ComputeFlight().
	193505	Structured aircraft properties returned by GetAircraft().
12.3	202851	Separate extra crew members into independent extra cabin and flight deck crew members in ComputeFlight().
	198296	Add Telvent wind source option for flight hazard handling in ComputeFlight().
	202297	STA becomes an input to ComputeFlight(), and ETA becomes a calculated value in the results.
12.5	207865	Add ability to generate errors from flight plan calculations that fail rule adherence in ComputeFlight().
	211216	Add option to NavWeb:FormatInput to request base-64-encoded return data, as well a flag identifying such content in NavWeb:FormatOutput, for use with ComputeFlight() and RecallFlight().
	213639	ICAO 2012 ATC strip generation option added to ComputeFlight().
	205279	Helicopter-only status included in output of GetSidStarList().
	211224	Add ability to identify a flight plan calculation as having been filed with an ATC itinerary in ComputeFlight().
	218195	Include ICAO 2012 configuration status to the output of GetAircraft().
	223153	Default reserve policy descriptions included in results from GetAircraft().
12.7	198293	Multi-leg flight brief retrieval support added to RecallFlight().
	235466	FormatId of empty string in NavWeb:FormatInput implies default format from aircraft or airline configuration.
13.1	224226	Allow flight level range requests in ComputeFlight().
	231182	Return list of performance profile keys from GetAircraft().
	229220	ATC callsign filing method control within ComputeFlight().

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

	222838	Minimum landing fuel based on time added to ComputeFlight() input fields and GetAircraft() response.
	240786	GetAlternateList() enhanced to filter results based on suitability.
	247218	Alternate maximum landing weight added to ComputeFlight().
13.3	206999	Calculation inputs based on template definitions specified in the Input Defaulter menu can be passed into ComputeFlight().
	222830	Delay cost control added to ComputeFlight().
	242286	Reclear and destination alternate route string specification within ComputeFlight().
	252310	Automatic ACM/BOW calculation from assigned crew members in ComputeFlight().
	287339	Flight paperwork banner text added to WeatherNotam().
14.1	254163	Alternate performance profile input added to ComputeFlight().
	275075	ComputeFlight() can be provided an event log string to provide meta-data about the calculation to troubleshooting staff.
	275353	Annotations added to WSDL for method and field descriptions.
	194314	RAIM validation option added to ComputeFlight().
14.3	267089	Instructor crew rank added to ComputeFlight().
	286054	Package template identifier added to NavWeb:FormatInput for use in ComputeFlight() and RecallFlight() when generating flight briefs.
	307726	Email subject field added to WeatherNotam().
15.1	276950	Point of safe return functionality added to ComputeFlight().
	295755	Add automatic next-ramp-required tankerage option to ComputeFlight().
	305126	MelCdl() method added.
15.3	291077	Second officer crew rank added to ComputeFlight().
	289961	Stored route comments returned from RouteAnalysis().
15.5	303640	Multi-leg tankering support added to ComputeFlight().
	327029	Add ImportFlight() call to create flights and assign pending flight plan calculation inputs.
15.7	284999	Allow callers to indicate that hold should be calculated over destination or alternate in ComputeFlight().
	348942	Optimized and ATC-compliant route generation option added to ComputeFlight().
16.1	326023	RecallFlight() can now accept an ETD offset value for the transmission time instead of specifying an absolute time.
	328849	ComputeFlight() can accept multiple additional fuels with reasons associated with them, as opposed to just one singular additional fuel value.
	229671	Climb, descent, and hold burn bias percentages added to ComputeFlight().
	336830	More complete rule adherence options added to ComputeFlight(), BuildRoute(), and RouteAnalysis().
	313285	RouteAnalysis() can return stored city-pair routes in unexpanded form.
	295894	GetFlightList() and GetFlightInfo() methods added.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

	275540	Icing protection method selection within ETP calculations in ComputeFlight().
	295882	Sparse compute support added to ComputeFlight().
	307125	Senior attendant crew rank added to ComputeFlight().
16.3	349268	RawWeatherNotam() method added.
	350215	Crew() method added.
	317971	Defaults() method added.
	361406	Wind source selection available within ComputeFlight().
	357213	Flight hazard avoidance (turbulence, icing, thunderstorms) in ComputeFlight() and BuildRoute().
16.5	357020	Individual flight deck and cabin crew weights available for use within ComputeFlight().
	359491	Omitting addresses from NavWeb:TransmitInput will result in the system automatically determining the appropriate destinations based on airline configuration.
	356834	Comments added to Defaults().
	336370	Alternate IFR fuel/time added to ComputeFlight().
	317970	Preferred payload added to ComputeFlight() and Defaults(), and requested ZFW added to Defaults().
	355791	Add APU time before and after refueling options to ComputeFlight().
	272843	Item10A and Item10B code overrides for ComputeFlight(), BuildRoute(), and RouteAnalysis().
	376683	FlightStatus() method added.
17.1	368181	Map layer GUIDs added as optional output within returns from ComputeFlight(), BuildRoute(), RouteAnalysis(), GetFlightInfo(), and RecallFlight().
	356897	Provide flight status properties in returns from GetFlightInfo().
	366220	Add traffic and purpose data from ICAO international NOTAM Q-line to output from RawWeatherNotam().
	374751	Add maximum flight level to GetAircraft() response.
	363842	Add ability to calculate the necessary ETD to meet the given STA within ComputeFlight().
	268319	Baggage handling added to ComputeFlight().
	374753	Add the analysis flight level, average wind component, and wind prognostication to routes returned by BuildRoute() and RouteAnalysis().
	374752	Preferred flight level override available for use in ComputeFlight(), BuildRoute(), and RouteAnalysis().
	374754	Allow RouteAnalysis() to only evaluate specific route types.
	375424	GetWxCharts() and GetWxChartInfo() methods added.
	384383	NavWeb:Format now accepts an indication that an ATC CHG message should be generated and transmitted, for use with ComputeFlight() and RecallFlight().
17.3	357606	Direct operating cost available as input to ComputeFlight(), BuildRoute(), and Defaults(), and delay cost configuration added to Defaults().

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

	357640	ComputeFlight() can calculate multiple PSRs by providing an ETP policy identifier.
	357751	NavWeb:Format now accepts an indication that an ATC DEP message should be generated and transmitted, for use with ComputeFlight() and RecallFlight().
	399769	GetServiceInfo() method added.
17.5	357807	ComputeFlight() and Defaults() can take a number of destination alternates to generate information for, and any in excess of the provided station identifiers can be dynamically selected and included on the flight plan.
	357615	Reserve fuel percentage can be explicitly provided to ComputeFlight() and Defaults().
	361570	Maximum fuel volume added to results of GetAircraft().
	386889	ATC, flight paperwork, and ACARS flight plan transmission status included in response from GetFlightInfo().
	357596	FlightStatus() can accept an automation profile name to indicate the automated behavior that should be applied.
	398336	Individual route legs returned from BuildRoute() and RouteAnalysis() include TAS, GS, wind component, as well as average and maximum wind shear.
	395438	To better support UFIs, an optional repeat number has been added to the NavWeb:FlightId structure, as well as an optional first leg origin date to the NavWeb:ComputeInput structure.
	405878	Minimum extra fuel lists have been incorporated into Defaults().
	294859	Navigator crew rank added to ComputeFlight().
	365595	The NavWeb:User structure now has an optional AIRAC cycle field that will control which navigation data set will be used for affected service calls.
	393010	Individual additional fuels can be marked as optional within the NavWeb:ComputeInput structure, in which case as much will be taken as possible, but an inability to accommodate all of it will not fail a flight plan calculation.
	408111	Restricted region avoidance in BuildRoute() and ComputeFlight() now supports disabled, warning, and error behavior akin to rules adherence, instead of the previous disabled/warning support.
	414609	Multi-leg tankerage airport goals can now be provided in Defaults().
	412370	Individual additional hold fuels can be marked as optional within the NavWeb:ComputeInput structure, in which case as much will be taken as possible, but an inability to accommodate all of it will not fail a flight plan calculation.
	360036	A default flight package template is now part of input defaults, and is usable within Defaults().
	414754	Future event-driven automation functionality can be toggled by input defaults, so a new field is now useable within Defaults().
17.7	387263	RawWeatherNotam() now makes query identifiers (stations, FIRs) optional, in addition to allowing requests to include inactive NOTAMs. These changes, combined with new NavWeb:Notam

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

		fields for cross-referencing allow this service call to provide data feed semantics by returning any changes since a given timestamp.
	385861	BuildRoute() now accepts numerous new input parameters to align its functionality with the N-FP user interface and control the generation of results, like RNP and permit checks, SID/STAR exclusion, and origin/destination runway selection methods.
	412768	A new RNP check value for BuildRoute() enables strict adherence.
	414027	Payload values can be passed to RouteAnalysis() to get a better estimate of the associated take-off weight and resulting likely cruising altitude to improve route optimization and more closely align estimates with flight plan calculation values.
	373163	A new comparison burn type has been added to the NavWeb:ComparisonInput structure, used by ComputeFlight() and ImportFlight(), which allows the specification of an ETOW delta to get a more accurate evaluation of the effect of take-off weight changes on fuel burn and enroute time.
	424284	RouteAnalysis() can take into account available permits to avoid airspace lacking permission to transit.
	422191	BuildRoute(), ComputeFlight(), and RouteAnalysis() take parameters to control whether user-defined restricted areas should be avoided.
	411913	The NavWeb:ComputeInput structure, used by ComputeFlight() and ImportFlight(), as well as the NavWeb:DefaultValuesInput structure, used by Defaults(), now provide an option to indicate that a flight plan calculation should consume its enroute reserves as part of the burn calculation.
	424518	BuildRoute() can accept a payload value or requested take-off weight to better determine the resulting likely cruising altitude to improve route optimization and more closely align estimates with flight plan calculation values.
	424576	RouteAnalysis() can accept a requested take-off weight to better determine the resulting likely cruising altitude to improve route optimization and more closely align estimates with flight plan calculation values. A maximum flight level can also be provided to limit the cruising altitude of results.
18.1	391125	Airlines now have the ability to specify routing rule identifiers that should be ignored in the route building and flight plan calculation stages. This behavior can be disabled by setting the ApplyRulesType field for BuildRoute(), ComputeFlight(), and RouteAnalysis() to ‘A’.
	414191	The CheckRnp field, used by BuildRoute(), ComputeFlight(), and RouteAnalysis(), changes the ‘S’trict value to ‘E’rror. A value of ‘S’ will continue to work, but it is recommended that clients change that logic to use ‘E’.
	433629	NavWeb:ExtraHoldInput now includes an optional UseTodBurnRate field, which when set to true will use that as opposed to a hold performance burn rate.
	362744	MZFW can now be specified within Defaults().

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

	390713	GetSidStarList() now returns independent boolean values for their applicability to jet aircraft, prop aircraft, or helicopters. Clients should migrate away from the use of the JetOnly, PropOnly, and HelicopterOnly flags now that multiple-applicability SID/STARs can be properly encoded.
	422277	RouteAnalysis() can now be instructed to return IFPS-compliant routes directly from the Eurocontrol CFMU/NM for intra-Europe flights.
	440086	ComputeFlight() can now be instructed to perform multi-leg tankerage for an explicit number of consecutive legs.
	429138	Response structures for all services now include execution time duration.
	401311	ComputeFlight() can now accept three NavWeb:ComparisonInput structures, instead of the previous limit of two.
18.3	411123	The alternate bias information, as well as fixed bias fuels and times for all phases of flight are now included in the response from GetAircraft().
	420938	SIDs/STARs now have the ability to be limited to certain aircraft types, or forbidden for certain aircraft types. This information is now returned as part of GetSidStarList() in the SidStarInfo structure.
	439533	SIDs/STARs now have the ability to be marked as domestic only, meaning only carriers considered domestic to the applicable airport's country may utilize it. This information is now returned as part of GetSidStarList() in the SidStarInfo structure.
	442693	BuildRoute(), ComputeFlight(), and RouteAnalysis() have a new option to keep an aircraft within ERA circles, meaning non-ETOPS-enabled aircraft will not venture more than the EEP/EXP time (60 minutes, by default) from an adequate airport.
	337794	GetWxCharts() now provides new request parameters to control the page size, scaling, and grayscale characteristics of the returned data.
18.5	452098	The NavWeb:OrbitInput structure now makes the FlHunFt field optional. This allows the system to dynamically determine the holding flight level based on the calculated cruising altitude of the main route, without the client application needing to explicitly specify a fixed value.
	451437	AddWeather() method added.
	452059	NavWeb:RouteAnalysisInput now has a new RouteLoadOnly parameter that will return results if available, but will not initiate a new route analysis if absent.
	460385	GetSupportedAirports() method added.
18.7	363065	New historical/statistical wind model and percentile selection in BuildRoute(), ComputeFlight(), RouteAnalysis(), and WindReport().
	438626	NOTAMs have had an FDC attribute added to identify them.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

	457670	ComputeFlight() and ImportFlight() can take lists of stations that should be excluded from system-chosen functionality, like automatic ETOPS and escape routes
	458338	EstimateFlightCost() method added.
	458544	ComputeFlight(), Defaults(), and ImportFlight() now accept customer IDs as used in the FOMS 575 station addresses database.
	461203	Airport information will now include any associated state code from FOMS 105.
	461880	Dead-head crew rank added to ComputeFlight().
	462272	RecallFlight() can now return EFF packages.
	462245	Observer crew rank added to ComputeFlight().
	460275	The NavWeb:ComputeInput structure allows the setting of an icing flag that will use an aircraft's icing driftdown performance for escape route and enroute terrain clearance calculations.
	470839	Calls to RouteAnalysis() can now specify the forecast wind source.
19.1	467486	BuildRoute() and RouteAnalysis() now take an optional scheduled trip time field as an input to provide more deterministic behaviour for functionality dependent on arrival time.
	472949	If the reclear waypoint is omitted in NavWeb:ComputeInput when a reclear airport is provided, the system will automatically select it.
	388245	A new hazard source field has been added to BuildRoute(), ComputeFlight(), and RouteAnalysis(). RouteAnalysis() has also had fields added to toggle avoidance of turbulence, icing, and storms.
19.3	480994	Sub-folders, topics, and titles can now be properly set and overridden via NavWeb:EffFormatInput for individual EFF components as generated by RecallFlight().
	480371	ComputeFlight() has had a Discard flag added which will prevent results from being saved to the database, leaving the state of the flight unchanged.
19.5	414845	Fuel prices can now have multiple values per station, specified by identifiers that are part of NavWeb:ComputeInput, used by ComputeFlight() and ImportFlight().
	472444	ComputeFlight() now takes a RaimCheck field that controls if/how RAIM coverage validation is done following a flight plan calculation.
	486408	ComputeFlight() now takes a AdsbCheck field that controls if/how ADS-B coverage validation is done following a flight plan calculation.
	480023	A new airport type attribute, indicating public or military use, has been added to the NavWeb:Airport structure.
	485027	Group membership information has been added to flights returned from GetFlightInfo().
	NFP-1714	Several new EFF item encoding types have been added to allow greater control over the file formats used by RecallFlight(),

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

		particularly for images. This also corrects erroneous MIME type values that were reported for various EFF components.
19.7	486408	ADS-B success/failure status enumerations have been added to the content returned by GetFlightInfo() and GetFlightList().
20.1	NFP-2271	The AllowEventDrivenAutomation field of the NavWeb:DefaultValuesInput structure is now deprecated and has no effect.
20.3	NFP-801	NOTAM requests made through RawWeatherNotam() will now return results for co-located airports, as well.
	NFP-2860	Position boundaries are now included for SIGMETs returned from RawWeatherNotam().
	NFP-2875	The Ids field of the NavWeb:RawWeatherNotamRequest object is now optional if querying only for SIGMETs. When combined with SigmetOffsetMin, from the NavWeb:RawWeatherInput structure, RawWeatherNotam() will return all SIGMETs issued in the associated last number of minutes.
	NFP-2989	Since SIGMETs can now be queried in bulk, instead of for particular FIRs, decoded SIGMET data now includes a list of all affected FIRs.
	NFP-3148	Crew members assigned to a flight as pending inputs are no longer lost when calling ComputeFlight() with the RecomputeLast flag enabled.
20.5	NFP-2955	Explicitly provided escape route stations can now be specified as part of the NavWeb:ComputeInput structure, used by ComputeFlight() and ImportFlight(). When given, these airports will override automatic selection criteria.
20.7	NFP-3632	The NavtechCrew.xsd schema, governing the content sent to Crew() has been re-annotated to provide more context, clarity, and information about functional behaviour.
	NFP-3794	The NavtechMelCdl.xsd schema, governing the content sent to MelCdl() has been re-annotated to provide more context, clarity, and information about functional behaviour.
	NFP-3657	EFF constraints, like the requirement for signing, can now be attached to documents specified in NavWeb:EffFormatInput structures, from which EFF archives can be generated via RecallFlight().
	NFP-3746	BuildRoute() and ComputeFlight() now have an IfpsAutoAssist flag that takes extra steps to generate a compliant flight plan for Eurocontrol airspace.
	NFP-3416	DecodeRoute() method added.
	NFP-1130	N-FP will more actively close connections from client applications that do not properly terminate sockets following responses, in order to enhance service availability and throughput.
	NFP-4035	GetFlightInfo() will now return the groups that flights are assigned to at the time of invocation, instead of the assignments at the time of the last flight plan calculation. This ensures that the results of GetFlightInfo() reflect the current state of the system, and also that non-computed flights will now show their associated groups.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

21.1	NFP-3375	IFR, VFR, and transitioning flight rules can now be specified, along with the positions of flight rule changes in calls to ComputeFlight().
	NFP-1195	Destination taxi time/fuel can now be provided in calls to ComputeFlight(), ImportFlight(), and Defaults().
21.3	S-53213	A new EtopsMinRadius field has been added to the NavWeb:ComputeInput structure, used by ComputeFlight() and ImportFlight(). When set to true, the system will select the smallest configured ETOPS radius for the applicable aircraft used in the calculation, which provides the necessary coverage of the route.
21.5	*N/A*	No changes.
21.7	S-60206	The NavWeb:AlternateInput structure now has a Rwy field that allows the explicit selection of a runway for alternate routings in calls to ComputeFlight().
	S-53690	NavWeb:ComputeInput has a new OverrideAptSuitability field, that when set to false will treat ARFF, crew qualification, minima, and suitability database errors as warnings in ComputeFlight().
	S-58007	A new AtcQueued enumerated value in the NavWeb:FlightStatusEnum type is now present, which will indicate that an ATC FPL submission has been performed through the EuroControl B2B services, but is queued for manual processing.
22.1	D-34520	When setting actuals with FlightStatus(), the FuelOnBoard value of the NavWeb:OooiValue structure will no longer be assumed to be in kilograms. It will now use the default weight unit configured for the environment, or can be overridden by the WeightUnit field of the same structure. NavWeb:OooiValue structures inside NavWeb:ActualFlightData returned by GetFlightInfo() will also now include the weight unit that the FuelOnBoard is represented in.
	D-30384	Omitting the TestFlight flag from the NavWeb:FlightListFilter structure given to GetFlightList() will now return both Normal and Test flights. Explicitly setting that field to true or false will return Test or Normal flights, respectively.
	D-34756	Uncomputed flights will now properly include any assigned groups in the output from GetFlightInfo().
22.3	S-54214	GetFlightList() will not be bound by the 1000 flight maximum return count when using the ForUtcDay filter criteria, such that an entire day's flights will be returned in that case.
22.5	D-36160	Fix internal tracking of Crew() webservice requests to ensure diagnostic information is associated with the correct customer airline code.
22.7	D-36863	Correct erroneous flagging of failure after calls to MelCdl()
	D-37255	Fix crash when calling WindReport() with a single waypoint.
	D-37559	Fix access to the GetSupportedAirports() method when accessed through the hosted service interface (see §4.2).

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

23.1	S-70268	An Arinc633Version field has been added to the NavWeb:RecallFlightRequest structure used by RecallFlight() to indicate which specification supplement should be used for the generation of ARINC 633 output.
	D-38099	Fix access to the EstimateFlightCost() method when accessed through the hosted service interface (see §4.2).
23.3	D-38300	Missing graphical weather charts will no longer result in an entire EFF build failure via RecallFlight(), and will instead create one with the content that was present, warning the caller of the missing components.
23.5	S-73171	BuildRoute(), ComputeFlight(), and RouteAnalysis() can now take an optional AcftEqpCheck parameter to produce lateral tracks and flight plans that acknowledge exemptions from things like CPDLC and ADS-B airspace requirements.
23.7	S-78057	WSDL annotations updated to list available values for aircraft equipment check parameters introduced with S-73171 in N-FP 23.5.
	D-40526	Allow route-leg fixed mach speed overrides to support both the presence and absence of an M prefix. (E.g. M80 vs. 80).
24.1	S-67435	The ApplyRulesType field, used as input to BuildRoute(), ComputeFlight(), and RouteAnalysis() can now take a value of ‘D’ to disable the RAD annex rules, and a value of ‘A’ to include all rules, even those marked as being ignored.
	S-78945	A new field, IfpsAutoAssistWithFls has been added for use in BuildRoute() and ComputeFlight() to automatically communicate with the Eurocontrol NM B2B services, like the existing IfpsAutoAssist parameter, except the new field will also retain the returned flight level profile information and have N-FP adhere to that if possible, instead of dynamically choosing its own.
24.3	S-62535	ATC flight number values are now returned from GetFlightInfo() and GetFlightList(), and ATC ALC values are also now returned by GetFlightInfo(). ATC ALC values are already returned by GetFlightList(), so that particular change was not necessary for this service call.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

3. Introduction

N-Flight Planning is a multi-tier service-oriented offering that provides a web-based user interface that is responsible for interacting with a powerful engine system to execute flight plan calculations and related dispatch activities.

While the N-FP UI is designed to maximize the benefit of flight planning and flight following functionality offered by the product, certain airline carriers and other providers of aviation services may find that NAVBLUE's web front-end does not provide the optimum solution given their workflows or usage patterns.

To that end, N-FP also offers an ever-growing set of webservices that provide programmatic access to flight plan functionality that can be directly offered to third-party systems. This allows NAVBLUE customers to create their own programs that utilize portions of N-FP functionality to provide solutions that meet their own unique needs.

3.1. Overview

Examples of applications that have already been created using these services include:

- Customized flight planning user interfaces.
- Weather/NOTAM pilot briefing modules.
- Server availability monitoring.
- Run-time performance measurement and metric collection.
- Automated viability determination for route contract bids.
- Tablet-based electronic flight bag applications.
- Third-party OCC suite integrations responsible for:
  - Aircraft maintenance.
  - Flight schedule.
  - Aircraft movement and assignment.
  - Crew roster and flight assignment.

The N-FP user interface communicates with the engine via its own SOAP communication channel, but this interface is private to NAVBLUE and not offered to third-party systems because:

- The NAVBLUE-specific interface is relatively volatile from an *API* perspective, changing with every version to match new functionality built into the product.
- N-Flight Planning, being a service-based offering that undergoes several significant updates annually, requires the flexibility and freedom to upgrade systems without dependencies on coordination with other providers.

To that end, a standalone set of backwards-compatible services were created to provide a stable foundation for other systems to make use of N-FP functionality in a reliable and forward-compatible way.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

3.2. History

The N-FP SOAP services are rooted in their support of NAVBLUE's Compass flight planning product.

Compass is a simple text message-based interface using comma-separated keyword and value pairs to provide flight plan inputs, such that it can respond with a human-readable operational flight plan.

This lightweight approach allows for easy incorporation into HTML web forms and aeronautical networks like SITA, ARINC, and AFTN in geographic areas with limited or unreliable communication infrastructure.

As NAVBLUE embarked on the creation of a brand new flight planning product with a more modern design philosophy and software architecture, the underpinnings of Compass also had to be adapted to work with the replacement system. To accomplish that, the SOAP services were created, and Compass was rewritten as the first client application that interacted with them.

Given that Compass revolved heavily around flight plan calculations and reporting tools that supported those operations, so too did the N-FP SOAP services in their infancy. Over time that functionality has grown, and continues to grow to include an increasingly diverse and rich set of dispatch-oriented functionality including systems integration, document transmission, and data retrieval.

Eventually, this functionality grew beyond the SOAP protocol to include access to N-FP's WMS-compliant geospatial engine for tile generation and flight plan data overlay.

3.3. Methodology

The N-FP SOAP services are developed with the following overarching design goals and approaches to problems:

- To provide 100% functional equivalency with the N-FP UI for flight plan calculations.
- To make available a loose set of SOAP data structures used in method calls such that they continue to generate schema-compliant *XML* for adjacent versions of the services.
- To tie into the same databases as the N-FP UI, such that customers and NAVBLUE support staff have the freedom to utilize different tools to access, manipulate, or troubleshoot the same flight information depending on situation or need.

Another important point to address is what the N-FP SOAP services are not. They are not designed to provide complete database access and manipulation for all modules available within the flight planning product. In most cases, the N-FP data maintenance tools remain the primary, if not sole method of system administration.

Certain data stores do have that level of control within these services (see §6), and those that do are typically common integration points between systems that round out the applications that comprise an operations suite, like crew tracking, maintenance management, or movement control.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

3.4. Migration

The need to be able to keep N-Flight Planning deployments on the same version was itemized in §3.1 and necessitates controlled changes to the webservice interface in order to facilitate flexible upgrades for client applications.

Evolutionary changes to input structures are done with new optional fields, such that older clients continue to generate compliant SOAP requests by virtue of those attributes simply being absent. Making use of new functionality, however, will eventually require updating client applications to populate that data.

Additions to output structures are done in two distinct phases in order to create a situation where a client compiled against the interface for a given N-FP version will work for the subsequent one, as well. In the first phase of a given functional change to webservice outputs, the N-FP WSDL definition includes new optional output fields for that major version that will not be populated. This allows clients to be updated to this interface and be prepared for those data elements in the future without introducing any functional changes. The following release implements the second phase, where the previous version's empty fields are finally populated and can be utilized by applications. This creates a buffer of several months for integrators to update their systems over the lifespan of an N-Flight Planning major release. It is important that integrators complete the updates to their system before the start of feature availability or they may start encountering errors within their application due to unexpected fields within the SOAP responses.

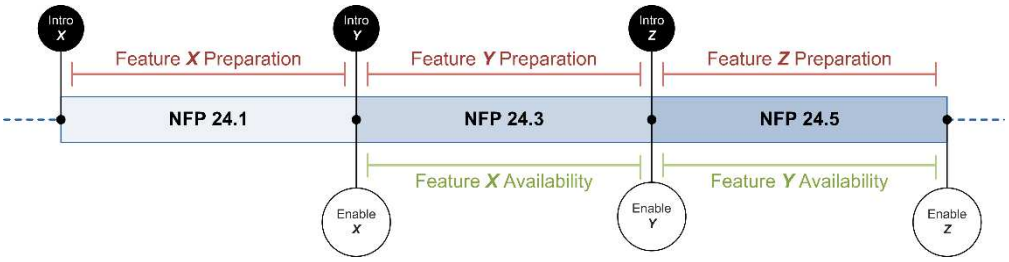

Figure 1: Two-Stage Output Field Addition

Importing new versions of the WSDL definition may also require slight changes to client code and recompilation, as API changes could occur that impose more structure from the source code perspective that still generate identical XML and satisfy backwards compatibility.

Information about specific changes warranting attention from integrators can be found in the returned data from `GetServiceInfo()` (see §6.14).

3.5. Designated Contacts

In order to ensure that the appropriate parties receive updated versions of this document and/or important information about the service offering, NAVBLUE maintains a list of designated technical contacts for third-party integrators that use N-Flight Planning's webservice functionality.

Please contact `support@navblue.aero` to assign or change this designated technical contact, and keep this information up to date as a safeguard against possible future service interruption.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

3.6. First Time Recipients

While all parts of this document are potentially relevant to integrators based on their particular client application needs, there are certain sections that are more generically applicable and important to all readers.

If you are receiving this document for the first time, it is highly suggested that you review and understand the content within the following sections:

- §1.2: Scope
- §1.9: Assumptions, Risks, and Mitigations
- §3.3: Methodology
- §3.4: Migration
- §3.5: Designated Contacts
- §4: Access
- §5.1: Rights Management
- §6.14: GetServiceInfo
- §8: SOAP Recipes

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

4. Access

There are two major deployment models for the N-Flight Planning system; a NAVBLUE-hosted service solution, and an in-house on-premises customer installation. The method by which the services are accessed will vary depending on the location of their servers and the associated network access they have.

4.1. In-House Installation

An in-house deployment involves having a set of NAVBLUE-managed servers installed within a customer's own network infrastructure. These typically sit within a *DMZ* such that NAVBLUE systems and staff can access them for monitoring, update, and troubleshooting purposes.

All NAVBLUE servers are generally accessible by the customer in this approach, allowing third-party applications to access the N-FP webservices directly on the engine server, provided that they also reside on the same network.

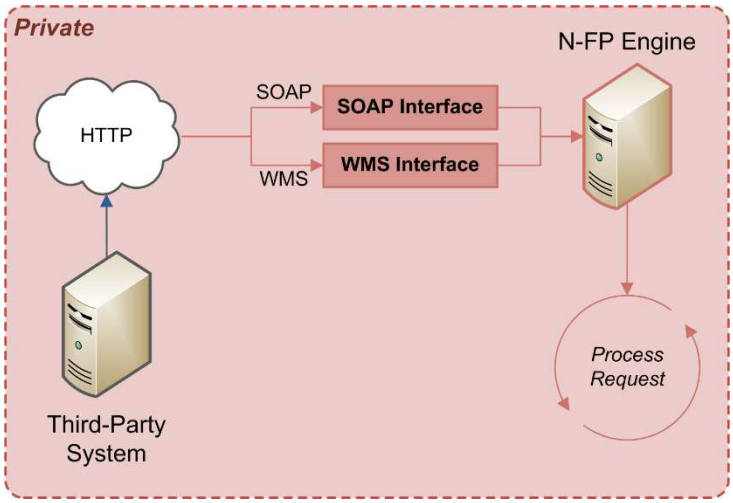

Figure 2: In-House Data Flow

The WSDL file matching the SOAP-offered functionality available on the installed N-FP version when using the in-house deployment model can be found at the following URL:

```
http://<ENGINE HOST>/wsdl/NavtechInterface.wsdl
```

The SOAP endpoint defined within that file simply needs to have the hostname replaced with the one that was used in the URL above, but the base URL for this access method is as follows:

```
http://<ENGINE HOST>:8018
```

For services utilizing N-FP's WMS functionality, the service is found at the following URL when using the in-house deployment model:

```
http://<ENGINE HOST>:13333/wms
```


Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

Information regarding the projections, layers, and query parameters that can be returned via calls to the WMS service can be retrieved in XML form by appending `?Request=GetCapabilities` to the CGI parameters provided to the URL above.

4.2. Hosted Service

A hosted service approach has a customer's flight planning environment present on either shared or dedicated hardware that resides within one of NAVBLUE's data centers.

Since the product is accessed via the Internet as a browser-based application, only the web server is exposed externally. The engine server sits protected behind a firewall whose only access provided to the outside world is granted to the N-FP web server. This approach can also be employed at customer-site installations to allow third-party integration capabilities from outside of their network.

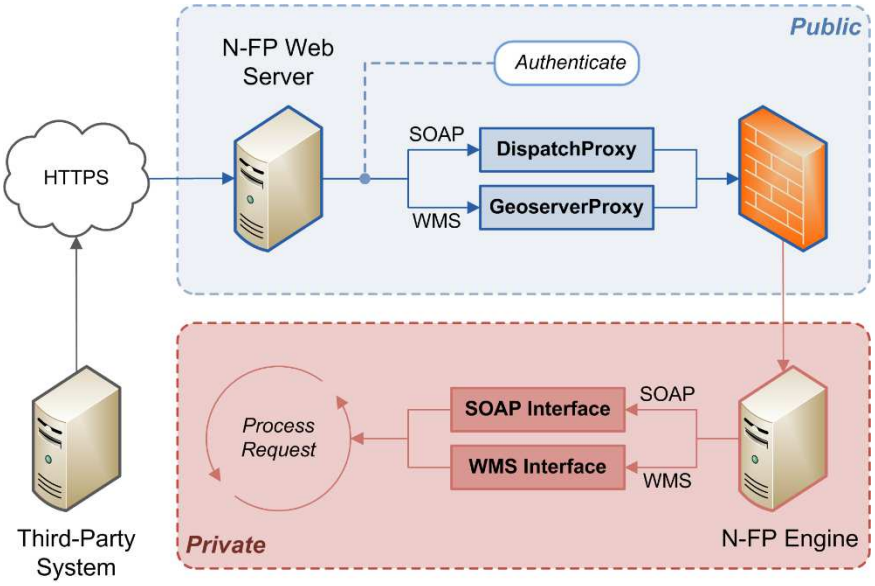

Figure 3: Hosted Service Data Flow

The WSDL file matching the SOAP-offered functionality available on the installed N-FP version when using the hosted service deployment model can be found at the following URL:

```
https://<WEB HOST>:444/NavtechPublicService.wsdl
```

The SOAP endpoint defined within that file simply needs to have the hostname replaced with the one that was used in the URL above, but the base URL for this access method is as follows:

```
https://<WEB HOST>:444/NavtechPublicService.svc
```

For services utilizing N-FP's WMS functionality, the service is found at the following URL:

```
https://<WEB HOST>/GeoserverProxyWeb/GeoserverProxy.axd
```

Information regarding the projections, layers, and query parameters that can be returned via calls to the WMS service can be retrieved in XML form by appending `?Request=GetCapabilities` to the CGI parameters provided to the URL above.

4.3. Motivation

The reason for the difference in access methods described in §4.1 and §4.2 is that the webservice functionality resides on the engine in both cases, but given the inability for hosted customers to access it, a proxy service is installed on the web server to forward requests to the engine in the protected environment, and shuttle responses back to the client. Hereafter this is referred to as the *DispatchProxy* for SOAP functionality, and the *GeoserverProxy* for WMS functionality.

4.4. Fail-Over

There are different circumstances under which an active customer environment may move from one system to another due to system failures or scheduled migration. This could take the form of movement from:

- A primary to warm-standby system for in-house deployments.
- An in-house system to a disaster recovery (*DR*) site, either customer-hosted or NAVBLUE-hosted.
- A NAVBLUE-hosted production system to a DR site.

It is important that users of N-Flight Planning's webservices build support into their applications and network infrastructure to configure and alter the endpoints utilized for accessing this functionality for cases when fail-over occurs. Such occurrences may take place with little-to-no notice based on operational impact. Note that movement to a DR site may involve changing the webservice access methodology from that described in §4.1 to the one covered in §4.2, depending on the system deployment approach.

It is also recommended that customers implement internal processes to perform the necessary communication, configuration, and modification of client application integrations when N-Flight Planning environment migrations take place. Operations and IT-related groups are typically disjoint and may not be universally aware of when a change occurs.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

5. Interface

5.1. Rights Management

There is no low-level rights management that controls access to the individual methods or pieces of functionality provided by the N-FP SOAP services. If a third-party system can access the engine, then it has access to the entire suite of methods therein. Access to individual portions of the service can be controlled by the third-party application to its users based on its own requirements.

When accessing the services via the DispatchProxy, however, a slight modification to the request structure needs to be made. Webservice access rights must be granted to a user within the N-FP User Administration Tool, and that user's authentication tokens (user name and password) must be included in the SOAP envelope to verify that they are able to access that customer's environment. This prevents a webservice user from accessing flight details for other customers that are hosted on the same server.

```
<s:Envelope
  xmlns:s="http://schemas.xmlsoap.org/soap/envelope/">
   <s:Header>
     <o:Security s:mustUnderstand="1"
       xmlns:o="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-
   wssecurity-secext-1.0.xsd">
       <o:UsernameToken>
         <o:Username>[USER NAME]</o:Username>
         <o:Password>[PASSWORD]</o:Password>
       </o:UsernameToken>
     </o:Security>
   </s:Header>

   <s:Body>
     <!--
       ...
       Message body here
       ...
     -->
   </s:Body>
</s:Envelope>
```

Note that ***[USER NAME]*** should be of the form *"ALC/user"*; such has `NAV/jdoe`, which would tell the system that the request should use the credentials of the John Doe user within the customer identified by the NAV airline code.

Passwords are transmitted as clear text, so the DispatchProxy is only served via HTTPS.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

5.2. XML Namespace

The data structures and methods within the N-FP SOAP services are contained in the `NavWeb` XML namespace.

5.3. NavWeb:User

All methods within the N-FP SOAP services take a `NavWeb:User` structure as part of their input request. This user information is not associated with authentication or authorization, and need not represent a user account within the N-FP User Administration Tool.

The purpose of this structure is three-fold. The first is the selection of the customer environment. Since a single N-FP system can host numerous self-contained and independent airlines, it is important that the `EnvAlc` field is populated with the three-letter ICAO airline code associated with the specific customer that the request is for. If in doubt, contact NAVBLUE for this information as it may not match the carrier airline code used for flights.

The second is tied exclusively to logging and can aid in troubleshooting activities, so providing a meaningful value is beneficial. The value passed to the N-FP SOAP services could be a user ID associated with the client application, or something more generic if it is used exclusively by a *B2B* service, for instance: `maintenancecontrol` or `briefingsystem`.

The final use is related to the optional selection of a particular *AIRAC* cycle for affected service calls. Those would include cases where navigation data is queried, such as:

- `GetAirport()`: See §6.10.
- `GetSidStarList()`: See §6.15

The other case is when used as an input to a flight plan calculation, namely:

- `ComputeFlight()`: See §6.3.
- `ImportFlight()`: See §6.19.

5.4. NavWeb:MapElements

The introduction of access to N-Flight Planning's WMS service creates a need to allow clients to request that route-specific elements be added onto rendered images. The different flight-specific elements are identified by *GUID*s, grouped in a `NavWeb:MapElements` structure. Requests to `BuildRoute()`, `ComputeFlight()`, `GetFlightInfo()`, `RecallFlight()`, and `RouteAnalysis()` can set `GetMapInfo` to `true`, which will result in this structure being passed back in the response. These identifiers can then be used with a `FILTERS` argument (see §7.1) for the `userdata` layer, described within the `GetCapabilities` information (see §4.1/§4.2).

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

5.5. NavWeb:FlightId

The early versions of the N-FP SOAP services uniquely identified flights by either a set of fields within a method's input structure, like `ComputeFlight()`, or as an integer serial number that represented both a flight and one of its particular calculation revisions, as used in `RecallFlight()`.

As new services continue to augment the SOAP offering, they frequently require one of these two methods to identify the flight they apply to.

With that in mind, a `NavWeb:FlightId` structure was created to encapsulate the notion of a flight identifier. Existing methods that used alternative means predating this structure will continue to work as before, but the use of `NavWeb:FlightId` where available is preferred since it represents the go-forward approach for future enhancements, and the former method is likely to be deprecated.

When writing client code that uses a service offering both methods, always utilize the `NavWeb:FlightId` structure, and consider retrofitting existing functionality to make use of it where available, since future enhancements will use it exclusively.

5.6. NavWeb:FormatInput

A key operation frequently executed by the SOAP services is the retrieval of generated output associated with a particular flight plan calculation, such as ATC filings, pilot briefings, and structured data in either standardized (e.g. ARINC 633 XML) or proprietary form (e.g. airline-specific XML, JSON, CSV, text).

The `NavWeb:FormatInput` structure defines these, which is utilized by `ComputeFlight()` (see §6.3) when immediately returning output following a successful flight plan calculation, or by `RecallFlight()` (see §6.22) when generating and/or transmitting output from a previously computed flight plan revision.

Only one of its optional content generation fields (those other than `EncodeAsPdf` and `Base64EncodeText`) may be populated in order to indicate what should be returned by the system, but the services that use it accept a list of `NavWeb:FormatInput` structures such that multiple products can be created within the context of a single method invocation.

Nearly all values for content fields can be determined through the use of the NFP user interface or consultation with those that utilize it within an airline operation. The WSDL annotations (see §4.1/§4.2 on how to access them) will provide details on what the fields correspond to.

Note that the use of the `PackageLayoutFile` field will likely require consultation with NAVBLUE in order to know what values can be utilized. This is because those values correspond to configuration files that can be unique to each N-FP system, such as customized integration output that is automatically generated upon activities like flight plan calculations, or package transmissions. There is no user-visible mechanism in place to determine those keys, so NAVBLUE technical staff can provide them in accordance with a client application's technical requirements.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

While there can be customer-specific values and output that can be produced on their behalf by NAVBLUE, there are a set of commonly-available options present on all N-FP systems for use with `PackageLayoutFile`, which follow below.

5.6.1. XML Output

The ARINC 633 specification defines an XML format for the common interchange of data across aviation services providers. The following keys are available on all customer installations since this is heavily standardized and common amongst all fo them.

```
Arinc633
Arinc633Airmet
Arinc633AirspaceData
Arinc633Atc
Arinc633CrewList
Arinc633Hazard
Arinc633Ntm
Arinc633Sigmet
Arinc633Supp3
Arinc633UpperAir
Arinc633Wx
```

5.6.2. Text & PDF Output

Given the common use case around extracting common flight brief components from N-FP for population in third-party platforms or EFF packagers, the following keys are available on all customer installations as well, and may represent text and/or richer PDF content. These identifiers will closely match the different available component types in the *Flight Package Layout* maintenance UI within the N-FP application.

```
integration_eff_aircraft_format
integration_eff_atc_form
integration_eff_brief
integration_eff_company_docs
integration_eff_company_notams
integration_eff_maps
integration_eff_notam_report
integration_eff_notam_table
integration_eff_route
integration_eff_route_validation
integration_eff_rwa
integration_eff_satellite
integration_eff_sigwx
integration_eff_tracks
integration_eff_weather
integration_eff_winds
```

5.7. NavWeb:EffFormatInput

Electronic Flight Folders (*EFF*) provide a standardized archive of documents and data that can be interpreted by a variety of Electronic Flight Bag (*EFB*) applications, whether connected to the aircraft or on a standalone device. The `NavWeb:EffFormatInput` structure defines each of the components to be included in that archive when generated through `RecallFlight()` (see §6.22).

This structure provides metadata fields used by the EFB shell to describe the items contained within, but the actual definition of content is done through the nested `NavWeb:FormatInput` structure (see §5.6, above).

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

The `FileNameTemplate` field accepts a string containing any combination of literal text or keywords that will be substituted with flight attributes when generating the resulting files. Note that the template can accept the same tokens as defined by the `strftime` standard library function to insert date and time information. Keywords are wrapped with square brackets to identify them, and a list follows below:

[ENVALC]	[WHOALC]	[ATCALC]
[FNUM]	[ORIG]	[DEST]
[ATC_FNUM]	[ORIG_IATA]	[DEST_IATA]
[EQUIP]	[SERIES]	[ENGINES]
[ACFT]	[SERIAL]	[FIR]
[ACFTREG]	[RECALL]	[TRIPNUM]
[DAY]	[DATE]	[TIME]
[SEQUENCE]	[CONTENT]	[MSGTYPE]

An example template string and associated output value follow:

```
[WHOALC][FNUM]_%Y-%m-%d_[ORIG]-[DEST]-[RECALL].pdf
NAV1234_2099-12-31_CYYZ-LFBO-3.pdf
```

5.8. Recall Numbers vs. Serial Numbers

Each independent flight plan calculation has two different numbers that help distinguish it from others; the recall number and serial number, each of which is part of the `NavWeb:Revision` structure, returned from `GetFlightInfo()` (see §6.12).

The recall number is an incrementing value, starting at 1, which increases for each flight plan calculation, and maintains the uniqueness of those numbers for just that flight such that there are no gaps in values.

Serial numbers also represent an incrementing value, but do so for any flight plan calculation done within a particular N-FP customer environment. That means that the serial number for the first plan calculated for a flight may appear to be completely random, and there will frequently be gaps in continuity as calculations are interspersed with those done different flights.

The figure below provides a visual depiction of how subsequent flight plan calculations result in assignment of both recall and serial numbers.

The left side illustrates how each recall number uniquely identifies a flight plan revision within the scope of a single flight, where as the right side shows how serial numbers contiguously increment across all flights for a customer.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

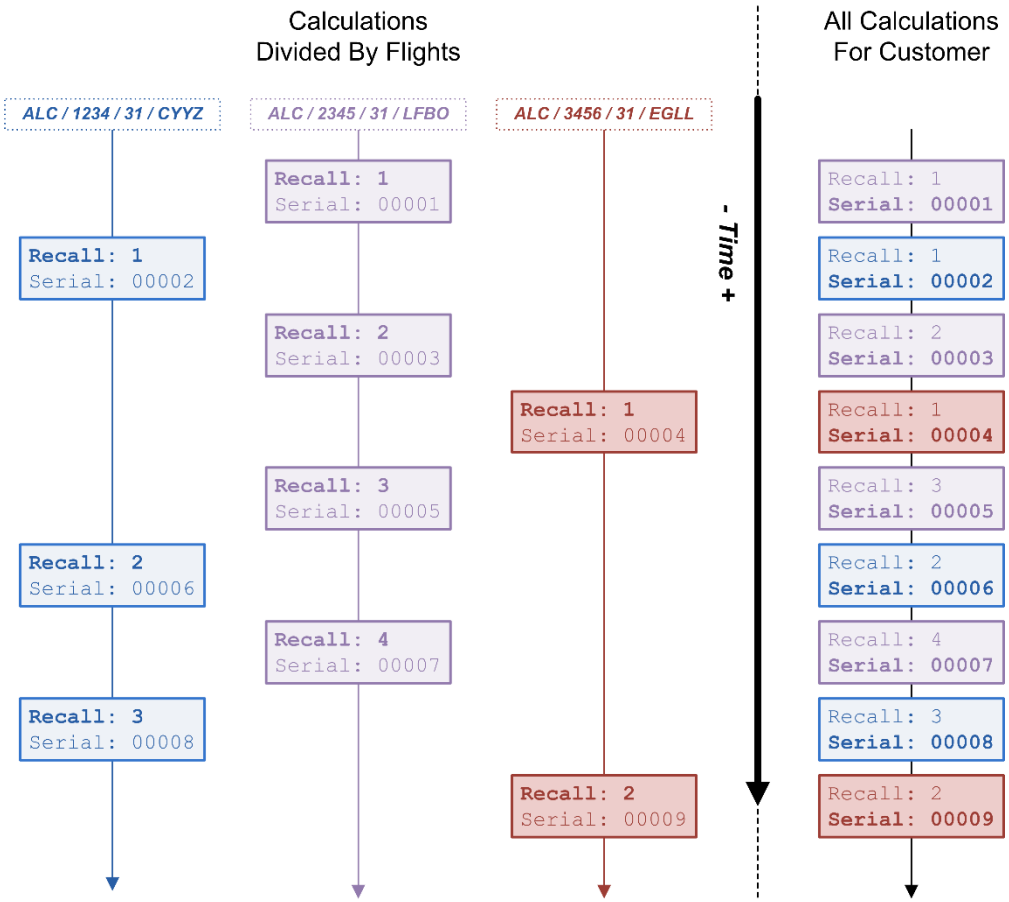

Figure 4: Recall vs. Serial Number Generation

Serial numbers are five digits by default, but can be configured to different lengths. The main reason for the existence of serial numbers is how they allow the system to concurrently reference both a particular flight and specific compute revision with just a single number, which is used with the `RecallFlight()` webservice call (see §6.22).

It is important to note that serial numbers roll over back to the start of the sequence when reaching the maximum number that can be represented with the configured length. For example, serial number 00001 follows 99999. This means that the uniqueness they offer is time bound, but generally on the scale of months, which is sufficient for N-FP's functional use cases.

6. SOAP Service Methods

A summary of each SOAP webservice method follows in the subsections below. As referenced in §1.9, the WSDL file may be more current than this document. The precise function signatures and data structures can also be found by referring to the WSDL file as indicated in §4.

In all cases, however, the method signatures use an input structure specific to that function, and return an associated response structure that is also specific to that function.

6.1. AddWeather

`AddWeather()` allows callers to insert *TAF*, *METAR*, and *SIGMET* bulletins into their systems, and optionally limit them to an ALC within their environment.

These weather products will subsequently be made available to all query functionality, such as `RawWeatherNotam()` (see §6.21), `RecallFlight()` (see §6.22), and `WeatherNotam()` (see §6.24), as well as the N-FP UI.

6.2. BuildRoute

`BuildRoute()` is responsible for taking a route string, formatted just like the input to the route expansion field on the N-FP UI's Route screen, and returns a structured response with details about that route, such as fix identifiers, geospatial position information, and estimated fuel burn.

The `Summary` field of the `NavWeb:Route` structure can be passed to the `ComputeFlight()` service (see §6.3) in the `RoutePath` field of the `NavWeb:ComputeInput` structure.

Setting `IfpsAutoAssist` to `true` in combination with the `RoutePath` field indicates that the system should automatically communicate with the Eurocontrol Network Manager in cases where either the inputted route, or the result of a dynamic route expansion does not pass their compliance check. N-FP will transparently retrieve suggested routes and return either the first compliant result, or the original candidate route along with a warning if no compliant route could be found.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

6.3. ComputeFlight

`ComputeFlight()` accepts a route and series of flight plan input parameters and returns an operational flight plan, and/or other representations/formats of computed data, such as parseable data structures like XML, JSON, and CSV. It also optionally provides a means to transmit documents from the system after successful calculations, like ACARS flight plan uplinks and ATC filings.

The `NavWeb:ComputeInput` structure is the most complicated within the N-FP SOAP services. There are few mandatory fields, with the vast majority being optional, even though at least a small subset of those optional fields must be provided.

The reason for this loosely-enforced input structure is the effect it has on forward-compatibility. By making any field that is not mandatory in all circumstances optional, new fields can be added and any existing client will continue to work. Fields can be deprecated as well, but they will remain present within the input structure for a lengthy period of time to ensure existing clients that used them will not result in unexpected XML schema violations upon method invocation. Deprecated field removal will be preceded by information via `GetServiceInfo()` (see §6.14) to allow sufficient time to mitigate those changes.

This flexibility comes at the cost of some API-level structure rigidity, whereby explicit choice selections would ensure that mutually exclusive input parameters cannot be sent in a single request.

Response structures will include error and/or warning information when invalid requests are made as part of their parameter verification activities. That can result in some extra development time and effort as clients are initially written to execute these services, but have little lasting overhead once complete.

Another item of special interest within the response from `ComputeFlight()` is the `SerialNumber` field. This uniquely identifies both a flight and a specific compute revision over a multi-month time frame, and is used as an input to `RecallFlight()` (see §6.22), or as a means to identify a flight within a `NavWeb:FlightId` structure for use with other methods (see §5.5).

`ComputeFlight()` also has the ability to provide sparse inputs, whereby only a small set of changes are provided such that the engine superimposes them on top of a previous compute revision's input. This allows a client application to only provide aspects of the flight that have changed since the last flight plan calculation instead of providing a complete set of input parameters.

This functionality is invoked under the following circumstances:

- If the `FlightId` of the provided `NavWeb:ComputeInput` structure includes a value for `SerialNumber` identifying the exact revision to build upon.
- If the `RecomputeLast` flag of the `NavWeb:ComputeFlightRequest` structure is set to `true`, in which case the most recent successful revision will be used.
- If the `RecomputeLast` flag of the `NavWeb:ComputeFlightRequest` structure is set to `true` but there are no successful calculations associated with the flight, in which case the most recent failed revision will be used.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

When `RecomputeLast` is set the system will also apply any databased pending inputs before integrating the provided sparse compute data. When providing a `SerialNumber` within `FlightId`, however, pending inputs are not applied and will end up discarded if the compute is successful.

One other aspect of the sparse compute input logic that bears mentioning are the side-effects that can occur to certain field inputs when particular values are set on others. N-FP has powerful automated default value settings based on carrier preferences to deal with unique operational circumstances, as well as its own handling of tightly linked flight plan inputs that cannot be done in isolation of each other. Extensive client testing dealing with changes of the following fields in sparse compute inputs is highly recommended to ensure expected operation:

- Destination
- Aircraft
- Route
- ALC

Other changes that can occur with a slightly lesser impact are fields that match keys of populated records in the Input Defaults database (FOMS 171).

There are fields within the `NavWeb:ComputeFlightRequest` structure that control certain subsequent actions after the completion of a successful flight plan calculation invoked with `ComputeFlight()`.

- `Discard`, when set to `true`, will ensure that no results are saved into the N-FP database, and that the state of the flight remains unchanged after execution. This can be helpful to run 'what-if' scenarios on a production flight to evaluate the effects of certain changes, without risking propagation into an operational scenario.
- `RaimCheck` and `AdsbCheck` will submit the resulting flight plan to third-party validation services. This may result in operational flight plan content detailing success or failure, and depending on the value given to these fields can have `ComputeFlight()` return errors when those checks are unsuccessful, or merely raise warnings.

Setting `IfpsAutoAssist` to `true` in the `NavWeb:ComputeInput` structure, in combination with the `RoutePath` , `RouteRank`, or `RouteCityPairNum` fields indicates that the system should automatically communicate with the Eurocontrol Network Manager in cases where the inputted route does not pass their compliance check. N-FP will transparently retrieve suggested routes and calculate the flight plan with either the first compliant result, or the original candidate route along with a warning if no compliant route could be found. Note that this is the only case where the result of a flight plan calculation has the potential to change an inputted value from the request. The `IfpsAutoAssist` functionality supplants the use of the `RouteOptimize` field.

It is also important to take care when setting weight values within the `NavWeb:ComputeInput` structure, as the units will vary depending on circumstance. Weight values will be interpreted to be in line with the instrumentation of any provided `AircraftId`, whether that be in pounds or kilograms. If an `AircraftId` is not provided, as part of a sparse compute, for instance, any weight values will be treated as a kilograms value.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

6.4. Crew

`Crew()` allows the caller to retrieve and modify flight operations personnel dictionary information, as accessed by *FOMS* 160 or the `Flight Operations Personnel` maintenance screen, as well as the individual assignments to flights.

The `Crew()` method is somewhat unique in that it does not take a regimented SOAP structure as its input parameter, and instead takes a raw entity-encoded or CDATA-enclosed XML string.

The reason is due to the complexity of operations that can be executed through this service, and the fact that this functionality can also be executed via file-based message passing.

An *XSD* schema governing the structure of the XML content can be retrieved by making a request to this service with the `GetXsdSchema` flag set to true in the `NavWeb:CrewRequest` structure. It will be returned in the `NavtechCrewXsdSchema` field of the `NavWeb:CrewResponse` structure.

Note that crew integration with N-FP can be difficult and more involved than other webservices given the range of functionality and relationships between rosters, qualifications, and flight assignments, as well as the associated complexity of the XSD that governs content sent to this service. A firm understanding of its behaviour within the flight planning system itself may be a significant prerequisite to working with `Crew()`. Functional subject matter experts may need to be consulted for overviews and/or training to support development in this area.

6.5. DecodeRoute

`DecodeRoute()` is responsible for taking a route string, formatted just like the input to the route expansion field on the N-FP UI's Route screen, and returns a structured response with fix identifiers and geospatial position information. It relies only on N-FP's internal navigation data, and allows callers to determine interim fixes on airways, SIDs, STARs, and organized tracks that are implied when defining just that start and end positions of their use.

Note that unlike the `BuildRoute()` service (see §6.2), `DecodeRoute()` does not take an aircraft identifier as an input, and thus does not do any optimized route expansion of dynamic vias identified by keywords like `MTTA`, `MTT`, or `MTTH`, nor does it provide estimates for enroute time or fuel burn.

The primary use case for `DecodeRoute()` is for third-party systems to properly capture the entire lateral track of a routing based on its ATC or condensed string representation, without access to an entire navigation data set, nor a completely configured flight planning environment, such that it can be displayed on *ASD* or EFB applications.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

6.6. Defaults

`Defaults()` allows the caller to retrieve and modify flight input default information, sometimes referred to as the "business rule" functionality of N-FP, as accessed via FOMS 171 or the `Input Defaults` maintenance screen.

Input defaults, specified by the customer, allow N-FP to automatically set various compute and route generation parameters based on flight characteristics, reducing the need for manual workload or dispatcher intervention.

Note that input defaulting functionality is a complicated topic within N-FP given its flexibility and ability to apply to a range of scenarios and use cases. A firm understanding of its behaviour within the flight planning system itself may be a significant prerequisite to working with `Defaults()`. Functional subject matter experts may need to be consulted for overviews and/or training to support development in this area.

6.7. EstimateFlightCost

Given a set of flight and aircraft related attributes, `EstimateFlightCost()` performs an analysis on a given lateral route track to generate estimated values for fuel, time, and costs, as well as overflight charges and distance information for each transited *FIR*, without performing a complete flight plan calculation. This is done without a flight key and results in no alterations of the system's state. There are a few different modes of operation:

- Providing an aircraft identifier will allow a complete cost analysis including fuel burn and estimated enroute time, but limits use to the databased state of aircraft within a given airline environment. Note that burn/time values will be considerably less accurate then those generated by `BuildRoute()` (see §6.2), which is in turn less accurate than those generated by `ComputeFlight()` (see §6.3).
- As an alternative to an aircraft identifier, a `NavWeb:AircraftAttrs` structure can be provided, which describes the aspects of aircraft that have an effect on overflight charge calculations. This allows client applications to dynamically alter those characteristics at the individual request level without any interaction with the aircraft database, but will result in no fuel or time information in the calculation results.
- The routing can be provided as either a city-pair route database number, a route string, or a list of waypoint identifiers, which can include pure latitude/longitude positions.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

6.8. FlightStatus

Given a flight identifier, `FlightStatus()` allows the caller to alter certain high-level characteristics of that flight in ways that are distinct from compute-related functionality.

Examples of the type of operations that can be performed are:

- Cancel and reinstate flights.
- Change the *STD* of a flight.
- Changing a flight between automated and manual states.
- Altering group membership of a flight.
- Setting *OOOI* time and/or fuel values.

6.9. GetAircraft

Given a fin or registration identifier, `GetAircraft()` returns details about the associated aircraft as specified in FOMS 220 or the `Aircraft Characteristics` maintenance screen.

6.10. GetAirport

Given an ICAO or IATA identifier, `GetAirport()` returns details about the associated aerodrome as specified in FOMS 105.

6.11. GetAlternateList

Given a station code, `GetAlternateList()` returns listings of alternates for that destination, as well as details about those airports as would be returned by `GetAirport()`. Results can also be filtered based on suitability as defined in FOMS 122 (`Airport Suitability`), as well as certain settings/values in FOMS 105, FOMS 170 (`Airline Code Parameters`), and FOMS 220 (`Aircraft Characteristics`). Consult the N-FP help documentation for more information on airport suitability.

6.12. GetFlightInfo

Given a list of flight identifiers, `GetFlightInfo()` returns details regarding those flights, both computed and pending (see §6.19), including applicable stations, aircraft, assigned crew, payload values, and a list of compute revision data.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

6.13. GetFlightList

Given a set of filter criteria, `GetFlightList()` returns a collection of flight summaries whose attributes match. Those criteria are contained within a `NavWeb:FlightListFilter` structure, and include:

- Components of the flight key, like airline codes and flight numbers.
- Station usage, in the form of the origin and destination aerodromes.
- Time constraints, like particular dates, or scheduled/estimated departure ranges.
- Other assigned flight attributes, such as crew, groups, and aircraft.

Flights returned by the service will adhere to all provided criteria, and not simply any of them.

6.14. GetServiceInfo

`GetServiceInfo()` returns the installed version and comments applicable to integrators regarding the N-Flight Planning webservices. The comments may include information about upcoming features, deprecated functionality removal, or potentially disruptive changes that need to be mitigated by client applications, such as those described in §3.4. It is advisable for users to examine the results of this method periodically.

6.15. GetSidStarList

Given a station code, `GetSidStarList()` returns SID and/or STAR information for that aerodrome, including both field values as specified in FOMS 320, and FOMS 325, as well as fix identifiers and geospatial position information.

6.16. GetSupportedAirports

Given an optional set of ICAO station identifiers, `GetSupportedAirports()` returns the associated ICAO/IATA identifiers and a timestamp indicating when the last update to those stations occurred. If the list of input stations is omitted, information on all airports within the N-FP system will be returned.

The main use case anticipated with this method is to provide a mechanism for third-party systems to synchronize airport information from N-FP by determining which stations' states are newer and then subsequently calling `GetAirport()` (see §6.10) to retrieve more details.

6.17. GetWxChartInfo

`GetWxChartInfo()` returns metadata about the various graphical weather charts available for retrieval from the system. This information is assembled into groups which hold similar chart content but with varying forecast times and altitudes. The chart data within each group contains listings of the applicable charts along with related information, in particular the `Id` field, which is given to `GetWxCharts()` (see §6.18).

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

6.18. GetWxCharts

Given a list of graphical weather chart identifiers provided by `GetWxChartInfo()` (see §6.17), `GetWxCharts()` returns a related list of associated base-64-encoded images, along with other information regarding the format of the image and the time it was received.

6.19. ImportFlight

`ImportFlight()` allows a caller to create flights and assign values to be taken into account during the next initiated flight plan calculation. It also allows supplemental information to be associated with the flight, like event logging details.

The same `NavWeb:ComputeInput` structure as used in `ComputeFlight()` (see §6.3) is taken as an argument, but no flight plan calculation is actually performed. The provided values within will be visible on the N-FP plan screen and part of the next compute, whether performed by a user or through automation. These are referred to as *pending inputs*, since they are associated with the flight, but have not yet been applied and incorporated into a computed flight plan.

Subsequent calls to `ImportFlight()` are superimposed on one another, so fields not explicitly set with a value will result in any existing settings being maintained instead of reset. For example, an initial call that sets a payload of 10,000 but nothing else, and a second call that only sets tankerage fuel to 1,000 will result in a set of pending compute inputs that includes both values. The fact that the payload was not included in the second call results in the existing value being untouched. The removal of a setting requires that an explicit value be assigned, for instance, using a payload value of 0 to remove a previously-assigned figure.

This allows multiple independent integrations to work seamlessly with N-FP without interfering with each other.

Unlike the sparse input functionality of `ComputeFlight()` (see §6.3) which is conditional on certain flags being present within the request structures, the inputs provided to `ImportFlight()` are always applied in a superimposed fashion as described above.

One special consideration to note are fields within the `NavWeb:ComputeInput` structure that are list types and the resulting complication involved in explicitly unsetting their values. The XML generated by the SOAP call does not allow for the distinction between an empty list and nothing specified at all, and it is always interpreted as the latter. This means that there is no way to explicitly remove any previously set list values, and they can only be replaced with other non-empty values.

Examples of fields that fall into this category are:

- Alternates.
- Comparison burns.
- Crew assignments.
- EEP/EXP and ETOPS stations.
- ETP pairs.
- Hold fuels.
- Orbit holds.
- Routing flight level requests.
- Routing performance profile overrides.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

Crew assignments are a special case within this list. Since all crew members across ranks are translated into a single data structure on the back-end service, providing any values for one rank (e.g. `CrewCaptain`) means that the absence of any other assignments to a different crew rank (e.g. `CrewFirstOfficer`) will result in their removal if already assigned to a given flight, since the aggregated-rank crew assignment structure being used as a replacement only includes the explicitly-provided set in the request structure. To avoid problems, when specifying crew assignments for any rank with `ImportFlight()`, be sure to specify all crew assignments with the request.

It is also important to take care when setting weight values within the `NavWeb:ComputeInput` structure, as the units will vary depending on circumstance. Weight values will be interpreted to be in line with the instrumentation of any provided `AircraftId`, whether that be in pounds or kilograms. If an `AircraftId` is not provided, any weight values will be treated as a kilograms value.

6.20. MelCdl

`MelCdl()` allows the caller to retrieve and modify master MEL/CDL lists as specified in FOMS 428 (`MEL CDL Dictionary`) for a given airframe, as well as the individual relationships to aircraft as specified in FOMS 425 (`MEL CDL Assignments`).

Like the `Crew()` method (see §6.4), `MelCdl()` is distinct from most other service calls in that it does not take a regimented SOAP structure as its input parameter, and instead takes a raw entity-encoded or CDATA-enclosed XML string.

The reason is due to the complexity of operations that can be executed through this service, and the fact that this functionality can also be executed via file-based message passing.

An *XSD* schema governing the structure of the XML content can be retrieved by making a request to this service with the `GetXsdSchema` flag set to true in the `NavWeb:MelCdlRequest` structure. It will be returned in the `NavtechMelCdlXsdSchema` field of the `NavWeb:MelCdlResponse` structure.

Note that MEL/CDL integration with N-FP can be difficult and more involved than other webservices given the range of functionality and relationships between dictionaries, performance penalties, and aircraft assignments, as well as the associated complexity of the XSD that governs content sent to this service. A firm understanding of its behaviour within the flight planning system itself may be a significant prerequisite to working with `MelCdl()`. Functional subject matter experts may need to be consulted for overviews and/or training to support development in this area.

6.21. RawWeatherNotam

`RawWeatherNotam()` gives the caller the ability to retrieve *WX/NOTAM* information, generally associated with a set of identifiers. The primary differentiator from `WeatherNotam()` (see §6.24) is in regards to the response, which provides structured data fields instead of a singular consolidated document.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

The request accepts parameterized configuration for weather bulletin retrieval, NOTAM retrieval, as well as optional *macro* strings to be expanded.

The response will include relevant data depending on the provided request parameters in the form of observations (METARs/*SPECI*s), forecasts (TAFs), significant weather (SIGMETs), NOTAMs, and macro reports.

Forecast and observation data will include decoded fields for the various time ranges enclosed within the related text bulletin. SIGMET data also includes polygon boundary coordinates.

Identifiers can be omitted from SIGMET queries provided that the `SigmetOffsetMin` field of the `NavWeb:RawWeatherInput` structure is set, and that the `ForecastCount` and `ObservationCount` fields are not. This allows the caller to get all SIGMETs issued in the last `SigmetOffsetMin` minutes.

NOTAM data can differ slightly in content compared to the N-FP application since the webservice call allows for the return of unfiltered records. A combination of customer-requested filters and upstream processing rules will remove data that NAVBLUE and the customer believe to be irrelevant to their operations. Greater control can be given to third-party integrators by allowing service requests to include this data anyway.

Identifiers can be omitted from NOTAM queries provided that the `ReceivedAfterTimeUtc` field of the `NavWeb:RawNotamInput` structure is set to a value within one day in the past of the current time. Combined with the `IncludeInactive` field and the `Action` and `Affected` attributes of the `NavWeb:Notam` structure, this provides data feed semantics yielding any changes from a particular point in time.

6.22. RecallFlight

`RecallFlight()` allows operations to be performed against a flight after it has been computed, either via `ComputeFlight()` or through the production N-FP user interface, such as returning an (or many) operational flight plan(s), transmitting messages, or generating electronic flight folders. There are three mechanisms by which content can be generated through different input fields:

- `Transmits`: Creates output based on the nested `NavWeb:FormatInput` structure (see §5.6) and immediately transmits that content to the specified addresses.
- `Formats`: Creates output based on the list of `NavWeb:FormatInput` structures provided (see §5.6) and returns the content in the response structure via the `Formats` field.
- `EffFormats`: Creates output based on the list of `NavWeb:EffFormatInput` structures provided (see §5.7) and returns the associated archive in the response structure via the `EffFileContents` field, and with a filename found in `EffFileName`.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

6.23. RouteAnalysis

`RouteAnalysis()` is responsible for evaluating the stored routes in FOMS 305 for a given city-pair, and possibly automatically-inserted routes by N-FP, by expanding dynamically-defined sections based on the provided inputs and meteorological data for the applicable time, and returning a ranked list of `NavWeb:Route` structures in descending order of benefit based on the desired optimization method (least cost, least fuel, or least time).

The returned list is a number of the same structures returned by `BuildRoute()`. The route provided to `ComputeFlight()` can be one of:

- `RouteRank`: The index into the ordered list.
- `RouteCityPairNum`: The explicit route identifier.
- `RoutePath`: The text string contained in the `Summary` field of the `Route` structure.

6.24. WeatherNotam

`WeatherNotam()` takes a list of stations and a route to create a WX/NOTAM brief that can be distributed to flight crews or other operational staff.

This service is unique in that it does not require a flight to be present in the N-FP system for it to be invoked, since all applicable input is included within the request structure.

6.25. WindReport

Given a list of positions, altitudes, a provider, and forecast time, `WindReport()` returns the wind direction, wind velocity, and outside air temperature for those positions.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

7. WMS Service Addendums

Detailed information about how to formulate WMS requests is addressed in §1.2 which references the URL of the master specification, and §4.1/§4.2, which indicate how to retrieve the service capabilities. There are, however, some NAVBLUE-specific interface additions that warrant special attention.

7.1. FILTERS

The `FILTERS` parameter is used to apply selection criteria to the items that would be rendered for a given layer. The exact filter parameters that are available are dependent on the layers to which they are associated with, and are documented within the service capabilities (see §4.1/§4.2).

Providing filters is done with a comma-separated list of arguments, bound by square brackets, and are indexed by the requested layers. An example follows:

`?Request=...&Layers=`*`L1,L2`*`&Filters=[`*`L1 Filters`*`],[`*`L2 Filters`*`]`

If there are no applicable filters for the layer in question, an empty set of square brackets is used, for example:

`?Request=...&Layers=`*`L1,L2`*`&Filters=[],[`*`L2 Filters`*`]`

7.2. INFOX / INFOY

Certain layers may have meta-data regarding the content, like validity times for many weather products. This information can be rendered on the resulting images at a location indicated by the `INFOX` and `INFOY` parameters. The values correspond to pixel offsets, where positive values are relative to the left and top of the image, and negative values are relative to the right and bottom of the image.

7.3. RADIUS

When using `GetFeatureInfo` to get information about the selected area of an image, `RADIUS` allows the caller to specify how large an area to consider, expressed in pixels. When unspecified, the default value is `5`. This only applies to lines and points since they can be difficult to select. It does not apply to polygon types.

7.4. WRAP

Setting `WRAP=TRUE` as a query string parameter allows the caller to retrieve an image that spans the international date line with a flat projection. Without this, the image will render the opposite content by capturing the geography in the opposite direction.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

7.5. QUERYTIME

Certain layer types, weather in particular, are sensitive to time given the fact that they typically encapsulate multiple forecasts into the future. Specifying `QUERYTIME` in the request allows the caller to control the applicable time that data should be retrieved for. It uses the format `YYYY-MM-DDTHH:MI:SS`, and is evaluated as a UTC time.

Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

8. SOAP Recipes

This section illustrates how to perform some common actions through N-FP's SOAP services by showing the request content that would be submitted. For brevity, only the content of the SOAP `<body>` tag is shown for each example.

8.1. Adding/Updating A Flight

Create flight `ALC/1234/31-Dec-1970/CYYZ` if it does not exist, then set the destination to `LFBO`, the aircraft to `N123NB`, and the payload to `5000` for that flight, whether newly created or pre-existing.

```
<NavWeb:ImportFlight>
  <Input xsi:type="NavWeb:ImportFlightRequest">
    <User xsi:type="NavWeb:User">
      <EnvAlc>ALC</EnvAlc>
      <UserId>user</UserId>
    </User>

    <Input xsi:type="NavWeb:ComputeInput">
      <FlightId xsi:type="NavWeb:FlightId">
        <Key xsi:type="NavWeb:FlightKey">
          <WhoAlc>ALC</WhoAlc>
          <FltNum>1234</FltNum>
          <StdUtc>1970-12-31T12:00:00</StdUtc>
          <OrigApt>CYYZ</OrigApt>
        </Key>
      </FlightId>

      <DestApt>LFBO</DestApt>
      <AircraftId>N123NB</AircraftId>
      <PayloadWgt>5000</PayloadWgt>
    </Input>
  </Input>
</NavWeb:ImportFlight>
```

8.2. Retrieve Service Version And Change Descriptions

There are no inputs necessary to retrieve important system version and upcoming functional changes, aside from the standard `NavWeb:User` structure.

```
<NavWeb:GetServiceInfo>
  <Input xsi:type="NavWeb:GetServiceInfoRequest">
    <User xsi:type="NavWeb:User">
      <EnvAlc>ALC</EnvAlc>
      <UserId>user</UserId>
    </User>
  </Input>
</NavWeb:GetServiceInfo>
```


Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

8.3. Generating Default Operational Flight Plan Format

Generate an operational flight plan using the airline/aircraft default format for the flight and compute revision that matches serial number `12345`.

```
<NavWeb:RecallFlight>
  <Input xsi:type="NavWeb:RecallFlightRequest">
    <User xsi:type="NavWeb:User">
      <EnvAlc>ALC</EnvAlc>
      <UserId>user</UserId>
    </User>

    <SerialNumber>12345</SerialNumber>

    <Formats xsi:type="NavWeb:FormatInput">
      <FormatId />
    </Formats>
  </Input>
</NavWeb:RecallFlight>
```

8.4. Generating ARINC 633 XML Flight Plan

Generate an XML document matching the ARINC 633 flight plan specification for the flight and compute revision that matches serial number `12345`.

```
<NavWeb:RecallFlight>
  <Input xsi:type="NavWeb:RecallFlightRequest">
    <User xsi:type="NavWeb:User">
      <EnvAlc>ALC</EnvAlc>
      <UserId>user</UserId>
    </User>

    <SerialNumber>12345</SerialNumber>

    <Formats xsi:type="NavWeb:FormatInput">
      <PackageLayoutFile>Arinc633</PackageLayoutFile>
    </Formats>
  </Input>
</NavWeb:RecallFlight>
```


Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

8.5. Generating Multiple Output Products

Generate the ARINC 633 XML output, a base-64-encoded PDF of the `International` flight brief layout, and the operational flight plan defined by format `50` for the flight and compute revision that matches serial number `12345`.

```
<NavWeb:RecallFlight>
  <Input xsi:type="NavWeb:RecallFlightRequest">
    <User xsi:type="NavWeb:User">
      <EnvAlc>ALC</EnvAlc>
      <UserId>user</UserId>
    </User>

    <SerialNumber>12345</SerialNumber>

    <Formats xsi:type="NavWeb:FormatInput">
      <PackageLayoutFile>Arinc633</PackageLayoutFile>
    </Formats>

    <Formats xsi:type="NavWeb:FormatInput">
      <EncodeAsPdf>true</EncodeAsPdf>
      <PackageTemplate>International</PackageTemplate>
      <Base64EncodeText>true</Base64EncodeText>
    </Formats>

    <Formats xsi:type="NavWeb:FormatInput">
      <FormatId>50</FormatId>
    </Formats>
  </Input>
</NavWeb:RecallFlight>
```


Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

8.6. Retieving MEL/CDL Dictionary Information

Retrieve all MEL/CDL dictionary items applicable to the A350-900 NFP equipment/series airframe, using XML entity encoding or CDATA enclosure.

```
<NavWeb:MelCdl>
  <Input xsi:type="NavWeb:MelCdlRequest">
    <User xsi:type="NavWeb:User">
      <EnvAlc>ALC</EnvAlc>
      <UserId>user</UserId>
    </User>

    <MelCdlRequestXml>
      &lt;MelCdlRequest&gt;
        &lt;GetMelCdl&gt;
          &lt;AircraftType&gt;
            &lt;Type&gt;
              &lt;Equipment&gt;A350&lt;/Equipment&gt;
              &lt;Series&gt;900&lt;/Series&gt;
            &lt;/Type&gt;
          &lt;/AircraftType&gt;

          &lt;Select&gt;&lt;All&gt;true&lt;/All&gt;&lt;/Select&gt;
        &lt;/GetMelCdl&gt;
      &lt;/MelCdlRequest&gt;
    </MelCdlRequestXml>
  </Input>
</NavWeb:MelCdl>
```

```
<NavWeb:MelCdl>
  <Input xsi:type="NavWeb:MelCdlRequest">
    <User xsi:type="NavWeb:User">
      <EnvAlc>ALC</EnvAlc>
      <UserId>user</UserId>
    </User>

    <MelCdlRequestXml>
      <![CDATA[
        <MelCdlRequest>
          <GetMelCdl>
            <AircraftType>
              <Type>
                <Equipment>A350</Equipment>
                <Series>900</Series>
              </Type>
            </AircraftType>

            <Select><All>true</All></Select>
          </GetMelCdl>
        </MelCdlRequest>
      ]]>
    </MelCdlRequestXml>
  </Input>
</NavWeb:MelCdl>
```


Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

8.7. Retrieving Flight Summaries Assigned To A Group/Desk

```
<NavWeb:GetFlightList>
  <Input xsi:type="NavWeb:GetFlightListRequest">
    <User xsi:type="NavWeb:User">
      <EnvAlc>ALC</EnvAlc>
      <UserId>user</UserId>
    </User>

    <Input xsi:type=”NavWeb:FlightListFilter>
      <ForDayUtc>1999-12-31</ForDayUtc>
      <GroupId>DOMESTIC</GroupId>
    </Input>
  </Input>
</NavWeb:GetFlightList>
```

8.8. Retrieving Basic Flight Information For Multiple Flights

```
<NavWeb:GetFlightInfo>
  <Input xsi:type="NavWeb:GetFlightInfoRequest">
    <User xsi:type="NavWeb:User">
      <EnvAlc>ALC</EnvAlc>
      <UserId>user</UserId>
    </User>

    <Ids xsi:type="NavWeb:FlightId">
      <Key xsi:type="NavWeb:FlightKey">
        <WhoAlc>ALC</WhoAlc>
        <FltNum>1234</FltNum>
        <StdUtc>1999-12-31T12:00:00Z</StdUtc>
        <OrigApt>CYYZ</OrigApt>
      </Key>
    </Ids>

    <Ids xsi:type="NavWeb:FlightId">
      <Key xsi:type="NavWeb:FlightKey">
        <WhoAlc>ALC</WhoAlc>
        <FltNum>5678</FltNum>
        <StdUtc>1999-12-31T14:35:00Z</StdUtc>
        <OrigApt>LFBO</OrigApt>
      </Key>
    </Ids>
  </Input>
</NavWeb:GetFlightInfo>
```


Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

9. WMS Recipes

This section illustrates how to perform some common actions through N-FP's WMS services by showing the request content that would be submitted. For clarity, only the URL query string is displayed, and parameters are wrapped to make them more easily readable. The actual URL to submit requests to can be found in §4.1/§4.2.

9.1. Showing Only Country Boundaries

At least one layer must be given to a request or else a blank image will be generated. Providing the `country` layer will return an image showing the geo-political boundaries within the bounding box.

```
?Request=GetMap
    &FORMAT=image/png
    &SRS=EPSG:4326
    &BBOX=-180,-90,180,90
    &HEIGHT=350
    &WIDTH=700
    &BGCOLOR=0xE0E5F3
    &LAYERS=country
    &STYLES=country
    &FILTERS=[]
```

9.2. Overlaying A Flight Plan Route

Graphical elements from a flight plan calculation are found in GUIDs returned within the `NavWeb:MapElements` structure (see §5.4). Adding these to an image involves adding another layer, style, and filter to the request.

```
?Request=GetMap
    &FORMAT=image/png
    &SRS=EPSG:4326
    &BBOX=-180,-90,180,90
    &HEIGHT=350
    &WIDTH=700
    &BGCOLOR=0xE0E5F3
    &LAYERS=country,userdata
    &STYLES=country,route_main
    &FILTERS=[],
             [guid IN ('85fe7040-8987-11ea-bf66-f01fafe47d95')]
```


Private & Confidential - This document is uncontrolled and therefore obsolete when printed.

9.3. Overlaying Multiple Routes

It is possible to add multiple flight plan element GUIDs onto a single layer. In the following example, two graphical routes are added with the `route_alternate` style, possibly both a take-off and destination alternate route.

```
?Request=GetMap
    &FORMAT=image/png
    &SRS=EPSG:4326
    &BBOX=-180,-90,180,90
    &HEIGHT=350
    &WIDTH=700
    &BGCOLOR=0xE0E5F3
    &LAYERS=country,userdata,userdata
    &STYLES=country,route_main,route_alternate
    &FILTERS=[],
             [guid IN ('85fe7040-8987-11ea-bf66-f01fafe47d95')],
             [guid IN ('85fe71f8-8987-11ea-bf66-f01fafe47d95',
                       '85fe7144-8987-11ea-bf66-f01fafe47d95')]
```

9.4. Overlaying Wind Barbs

Non-flight-specific elements can also be overlaid, the exact capabilities and parameters for which can be found in the `GetCapabilities` call (see §4.1/§4.2).

```
?Request=GetMap
    &FORMAT=image/png
    &SRS=EPSG:4326
    &BBOX=-180,-90,180,90
    &HEIGHT=350
    &WIDTH=700
    &BGCOLOR=0xE0E5F3
    &LAYERS=country,userdata,userdata,winds
    &STYLES=country,route_main,route_alternate,winds
    &FILTERS=[],
             [guid IN ('85fe7040-8987-11ea-bf66-f01fafe47d95')],
             [guid IN ('85fe71f8-8987-11ea-bf66-f01fafe47d95',
                       '85fe7144-8987-11ea-bf66-f01fafe47d95')],
             [SRC=NWS AND FL=340 AND DTG=2912]
```


Private & Confidential - This document is uncontrolled and therefore obsolete when printed.